

École des Ponts

ParisTech

Robust CVRP - Modélisation Papier

Projet d'optimisation robuste - MPRO

Nikita-Marc Pomiès
Rayan Abderrezak Hamoud

Paris, Novembre 2024

Contents

1	Modélisation papier	1
1.1	Modélisation Statique	1
1.2	Modélisation Robuste	2
1.3	Résolution par plans coupants et LazyCallback	2
1.4	Résolution par dualisation	4
2	Autre formulation du problème pour faire face aux symétries	6
2.1	Modélisation polynomiale par les flots	6
2.2	Introduction de la notion de Peak Customer	6
2.3	Formulation du problème	7
2.4	Contraintes supplémentaires permettant de resserrer la relaxation linéaire	7
2.5	Application au problème robuste	7
3	Résolution numérique	9
3.1	Plans coupants	9
3.2	Dualisation	9
3.3	Métaheuristique	10
3.4	Génération de colonnes	13
4	Résultats expérimentaux	15
4.1	Petites Instances ($n \leq 30$)	15
4.2	Grandes instances	17
4.3	Retour sur la méthode de génération de colonnes	18

1 | Modélisation papier

1.1 | Modélisation Statique

Nous partons de la modélisation PLNE classique du problème du Capacitated Vehicle Routing

$$\min_x \sum_{(i,j) \in A} t_{ij} x_{ij} \quad (1.1)$$

$$\text{s.t } \sum_{i \in [n]} x_{ij} = 1 \quad \forall j \in [2, n] \quad (1.2)$$

$$\sum_{i \in [n]} x_{ji} = 1 \quad \forall j \in [2, n] \quad (1.3)$$

$$\sum_{i \notin S} \sum_{j \in S} x_{ij} \geq \sum_{j \in S} d_j \quad \forall S \subseteq V - \{1\}, S \neq \emptyset \quad (1.4)$$

$$\sum_{i \in S} \sum_{j \in S} x_{ij} t_{ij} \leq C \quad \forall S \subseteq V - \{1\}, S \neq \emptyset \quad (1.5)$$

$$x_{ij} \in \{0, 1\} \quad \forall (i, j) \in A \quad (1.6)$$

Cette formulation comporte un nombre exponentiel de contraintes (1.4) et (1.5).

Il est possible d'obtenir une formulation compacte en utilisant une famille de contraintes polynomiales (appelées contraintes MTZ).

$$\min_x \sum_{(i,j) \in A} t_{ij} x_{ij} \quad (1.7)$$

$$\text{s.t } \sum_{i \in [n]} x_{ij} = 1 \quad \forall j \in [2, n] \quad (1.8)$$

$$\sum_{i \in [n]} x_{ji} = 1 \quad \forall j \in [2, n] \quad (1.9)$$

$$w_i - w_j \leq C(1 - x_{ij}) - d_j \quad \forall (i, j) \in A \quad (1.10)$$

$$d_i \leq w_i \leq C \quad \forall i \in [n] \quad (1.11)$$

$$x_{ij} \in \{0, 1\} \quad \forall (i, j) \in A \quad (1.12)$$

La variable w_i correspond au chargement restant dans le camion avant son passage par le sommet i . Si $x_{i,j} = 0$, la contrainte (1.10) n'est pas active et s'obtient naturellement de (1.12). Si $x_{i,j} = 1$, on obtient $w_i + d_j \leq w_j$ qui caractérise bien que l'on a assez de vaccins en arrivant vers le client j . Cette contrainte empêche également la formation de sous tours admissibles.

1.2 | Modélisation Robuste

Détaillons la modélisation du problème robuste, avec un ensemble d'incertitude sur les durées des trajets entre les sommets V , noté $\mathcal{U}$.

On a

$$\mathcal{U} = \{t'_{ij} = t_{ij} + \delta_{ij}^1(\hat{t}_i + \hat{t}_j) + \delta_{ij}^2 \hat{t}_i \hat{t}_j \text{ tel que} \\ \sum_{(i,j) \in A} \delta_{ij}^1 \leq T, \sum_{(i,j) \in A} \delta_{ij}^2 \leq T^2, \delta_{ij}^1 \in [0, 1], \delta_{ij}^2 \in [0, 2] \forall (i, j) \in A\}$$

Le problème de CVRP robuste peut donc alors s'écrire avec la formulation PLNE mixte suivante:

$$\min_x \max_{\delta^1, \delta^2} \sum_{(i,j) \in A} t_{ij} x_{ij} + (\delta_{ij}^1(\hat{t}_i + \hat{t}_j) + \delta_{ij}^2 \hat{t}_i \hat{t}_j) x_{ij} \quad (1.13)$$

$$\text{s.t } \sum_{i \in [n]} x_{ij} = 1 \quad \forall j \in [2, n] \quad (1.14)$$

$$\sum_{i \in [n]} x_{ji} = 1 \quad \forall j \in [2, n] \quad (1.15)$$

$$w_i - w_j \leq C(1 - x_{ij}) - d_j \quad \forall (i, j) \in A \quad (1.16)$$

$$d_i \leq w_i \leq C \quad \forall i \in [n] \quad (1.17)$$

$$\sum_{(i,j) \in A} \delta_{ij}^1 \leq T \quad (1.18)$$

$$\sum_{(i,j) \in A} \delta_{ij}^2 \leq T^2 \quad (1.19)$$

$$x_{ij} \in \{0, 1\} \quad \forall (i, j) \in A \quad (1.20)$$

$$\delta_{ij}^1 \in [0, 1], \delta_{ij}^2 \in [0, 2] \quad \forall (i, j) \in A \quad (1.21)$$

1.3 | Résolution par plans coupants et LazyCallback

a)

Soit $\mathcal{X}$, l'ensemble des x vérifiant les contraintes du problème communes à la formulation du CVRP statique. Le problème robuste se réécrit alors :

$$\min_x \max_{t' \in \mathcal{U}} \sum_{(i,j) \in A} t'_{ij} x_{ij} \quad (1.22)$$

$$x \in \mathcal{X} \quad (1.23)$$

Nous souhaitons faire apparaître la robustesse uniquement dans les contraintes. Le problème précédent se réécrit alors :

$$\min_{x, Z} Z \quad (1.24)$$

$$\text{s.t } \sum_{(i,j) \in A} t'_{ij} x_{ij} \leq Z \quad \forall t' \in \mathcal{U} \quad (1.25)$$

$$x \in \mathcal{X} \quad (1.26)$$

b)

Pour résoudre ce problème avec l'algorithme des plans coupants, nous devons définir un ensemble $\mathcal{U}^*$ initial de scénarios (restreint par rapport à la taille de $\mathcal{U}$)

Nous avons pris $\mathcal{U}^* = \{t_{ij} | (i, j) \in A\}$ qui revient à prendre les durées du problème statique. L'implémentation pratique pourra nous amener à en changer. Nous pourrions également prendre un ensemble initial de scénario vide.

c)

A chaque itération de l'algorithme de plan coupant, il faut résoudre le sous problème suivant (x^* est alors un paramètre du sous problème):

$$\max_{t' \in \mathcal{U}} \sum_{(i,j) \in A} x_{ij}^* t'_{ij} \quad (1.27)$$

Détaillons clairement les contraintes portant sur les variables δ^1, δ^2 .

$$\max_{\delta^1, \delta^2} \sum_{(i,j) \in A} t_{ij} x_{ij}^* + (\delta_{ij}^1 (\hat{t}_i + \hat{t}_j) + \delta_{ij}^2 \hat{t}_i \hat{t}_j) x_{ij}^* \quad (1.28)$$

$$\text{s.t.} \sum_{(i,j) \in A} \delta_{ij}^1 \leq T \quad (1.29)$$

$$\sum_{(i,j) \in A} \delta_{ij}^2 \leq T^2 \quad (1.30)$$

$$\delta_{ij}^1 \in [0, 1], \delta_{ij}^2 \in [0, 2] \quad \forall (i, j) \in A \quad (1.31)$$

La résolution de ce problème nous donne donc δ^{1*}, δ^{2*} .

d)

Après avoir résolu le sous-problème, nous vérifions si x^* et Z^* les solutions de (MP) vérifient le scénario donnée par $(\delta^{1*}, \delta^{2*})$

$$\sum_{(i,j) \in A} t_{ij} x_{ij}^* + (\delta_{ij}^{1*} (\hat{t}_i + \hat{t}_j) + \delta_{ij}^{2*} \hat{t}_i \hat{t}_j) x_{ij}^* \leq Z^*$$

Si c'est le cas, x^*, Z^* sont les solutions optimales du CVRP robuste étudié.

e)

Si x^*, Z^* ne satisfont pas le scénario le plus dégradant (obtenu par résolution du sous-problème), on doit ajouter ce scénario au problème maître. Le sous problème ajoute donc des coupes de la forme :

$$\sum_{(i,j) \in A} t_{ij} x_{ij} + (\delta_{ij}^{1*} (\hat{t}_i + \hat{t}_j) + \delta_{ij}^{2*} \hat{t}_i \hat{t}_j) x_{ij} \leq Z$$

où δ^{1*}, δ^{2*} sont les solutions optimales du sous-problème.

On actualise alors $\mathcal{U}^* \leftarrow \mathcal{U}^* \cup \{t' = (t_{ij} + \delta_{ij}^{1*} (\hat{t}_i + \hat{t}_j) + \delta_{ij}^{2*} \hat{t}_i \hat{t}_j)_{i,j \in A}\}$.

1.4 | Résolution par dualisation

a)

On réécrit le problème en isolant les variables δ^1, δ^2 .

$$\min_x \sum_{(i,j) \in A} t_{ij} x_{ij} + \eta(x) \quad (1.32)$$

$$\text{s.t.} \sum_{i \in [n]} x_{ij} = 1 \quad \forall j \in [2, n] \quad (1.33)$$

$$\sum_{i \in [n]} x_{ji} = 1 \quad \forall j \in [2, n] \quad (1.34)$$

$$w_i - w_j \leq C(1 - x_{ij}) - d_j \quad \forall (i, j) \in A \quad (1.35)$$

$$d_i \leq w_i \leq C \quad \forall i \in [n] \quad (1.36)$$

$$x_{ij} \in \{0, 1\} \quad \forall (i, j) \in A \quad (1.37)$$

où

$$\eta(x) = \max_{\delta^1, \delta^2} \sum_{(i,j) \in A} [\delta_{ij}^1 (\hat{t}_i + \hat{t}_j) + \delta_{ij}^2 \hat{t}_i \hat{t}_j] x_{ij} \quad (1.38)$$

$$\text{s.t.} \sum_{(i,j) \in A} \delta_{ij}^1 \leq T \quad (1.39)$$

$$\sum_{(i,j) \in A} \delta_{ij}^2 \leq T^2 \quad (1.40)$$

$$\delta_{ij}^1 \leq 1 \quad \forall (i, j) \in A \quad (1.41)$$

$$\delta_{ij}^2 \leq 2 \quad \forall (i, j) \in A \quad (1.42)$$

$$\delta_{ij}^1, \delta_{ij}^2 \geq 0 \quad \forall (i, j) \in A \quad (1.43)$$

b)

Le problème de maximisation interne est donné par η :

$$\max_{\delta^1, \delta^2} \sum_{(i,j) \in A} [\delta_{ij}^1 (\hat{t}_i + \hat{t}_j) + \delta_{ij}^2 \hat{t}_i \hat{t}_j] x_{ij} \quad (1.44)$$

$$\text{s.t.} \sum_{(i,j) \in A} \delta_{ij}^1 \leq T \quad (1.45)$$

$$\sum_{(i,j) \in A} \delta_{ij}^2 \leq T^2 \quad (1.46)$$

$$\delta_{ij}^1 \leq 1 \quad \forall (i, j) \in A \quad (1.47)$$

$$\delta_{ij}^2 \leq 2 \quad \forall (i, j) \in A \quad (1.48)$$

$$\delta_{ij}^1, \delta_{ij}^2 \geq 0 \quad \forall (i, j) \in A \quad (1.49)$$

c)

Calculons le dual du problème de maximisation interne :

$$\min_{\lambda, \mu} \lambda_1 T + \lambda_2 T^2 + \sum_{(i,j) \in A} \mu_{ij}^1 + 2 \sum_{(i,j) \in A} \mu_{ij}^2 \quad (1.50)$$

$$\text{s.t.} \mu_{ij}^1 + \lambda_1 \geq (\hat{t}_i + \hat{t}_j) x_{ij} \quad \forall (i, j) \in A \quad (1.51)$$

$$\mu_{ij}^2 + \lambda_2 \geq \hat{t}_i \hat{t}_j x_{ij} \quad \forall (i, j) \in A \quad (1.52)$$

$$\mu_{ij}^1, \mu_{ij}^2 \geq 0 \quad \forall (i, j) \in A \quad (1.53)$$

$$\lambda_1, \lambda_2 \geq 0 \quad (1.54)$$

d)

Notre problème de CVRP robuste est donc équivalent au PLNE suivant :

$$\min_{x, \lambda, \mu} \sum_{(i,j) \in A} t_{ij} x_{ij} + \lambda_1 T + \lambda_2 T^2 + \sum_{(i,j) \in A} \mu_{ij}^1 + 2 \sum_{(i,j) \in A} \mu_{ij}^2 \quad (1.55)$$

$$\text{s.t.} \sum_{i \in [n]} x_{ij} = 1 \quad \forall j \in [2, n] \quad (1.56)$$

$$\sum_{i \in [n]} x_{ji} = 1 \quad \forall j \in [2, n] \quad (1.57)$$

$$w_i - w_j \leq C(1 - x_{ij}) - d_j \quad \forall (i, j) \in A \quad (1.58)$$

$$d_i \leq w_i \leq C \quad \forall i \in [n] \quad (1.59)$$

$$\mu_{ij}^1 + \lambda_1 \geq (\hat{t}_i + \hat{t}_j) x_{ij} \quad \forall (i, j) \in A \quad (1.60)$$

$$\mu_{ij}^2 + \lambda_2 \geq \hat{t}_i \hat{t}_j x_{ij} \quad \forall (i, j) \in A \quad (1.61)$$

$$\mu_{ij}^1, \mu_{ij}^2 \geq 0 \quad \forall (i, j) \in A \quad (1.62)$$

$$\lambda_1, \lambda_2 \geq 0 \quad (1.63)$$

$$x_{ij} \in \{0, 1\} \quad \forall (i, j) \in A \quad (1.64)$$

2 | Autre formulation du problème pour faire face aux symétries

Lorsque nous considérons les instances euclidiennes, nous sommes confrontés au fait que toute solution, identifiée par un tout dépôt-clients-dépôt peut être représentée de la même manière par le tour effectué dans le sens inverse. Nous nous sommes rendus compte de ce problème, lorsque nous souhaitons résoudre les instances euclidiennes par la méthode des plans coupants, mais globalement le problème apparaît dès lors que l'on effectue un branch and bound. En effet, on obtient des solutions équivalentes si on inverse les chemins individuels dans une solution du problème et ça implique un nombre exponentielle (dans le nombre de routes disjointes) de solutions équivalentes. Nous avons donc recherché une formulation qui casse la symétrie. Nous nous sommes donc appuyés sur les travaux de (Gadegard and al.)[1] qui partent d'une formulation polynomiale du problème du CVRP faisant intervenir des flûts entre les sommets, et qui introduisent par la suite la notion de "peak customer". Ces clients permettront d'identifier de manière unique une route incluse dans une solution du problème.

2.1 | Modélisation polynomiale par les flûts

Le point de départ du papier est cette modélisation du CVRP : On note $A = (i, j), i \in [1, n], j \in [1, n]$ et $V_c = [2, n]$ les sommets représentant les clients, d_i les demandes des clients et C la capacité d'un camion.

$$\min \sum_{(i,j) \in A} t_{ij} x_{ij} \quad (1)$$

$$\text{s.t.} \quad \sum_{j \in V_c \cup \{1\}} x_{ji} = 1, \quad \forall i \in V_c \quad (2)$$

$$\sum_{j \in V_c \cup \{1\}} x_{ij} = 1, \quad \forall i \in V_c \quad (3)$$

$$\sum_{j \in V_c \cup \{1\}} f_{ij} = \sum_{j \in V_c \cup \{1\}} f_{ji} + d_i, \quad \forall i \in V_c \quad (4)$$

$$d_i x_{ij} \leq f_{ij} \leq (C - d_j) x_{ij}, \quad \forall (i, j) \in A \quad (5)$$

$$x_{ij} \in \{0, 1\}, f_{ij} \in \mathbb{N} \quad \forall (i, j) \in A \quad (6)$$

La variable de flot f_{ij} représente ici la quantité totale livrée sur la route lorsque le camion quitte le client i . Nous avons implémenté ce modèle, en ajoutant une contrainte sur un nombre minimal d'itinéraires $\sum_{j \in V_c} x_{1j} \geq \frac{1}{C} \sum_i d_i$ qui permet d'accélérer la relaxation. Il est à noter que dans l'article, les auteurs s'intéressent au CVRP avec un nombre fixé de camions.

Cette formulation conserve la symétrie des solutions car une route pour un véhicule donné est toujours décrite par un cycle dépôt-clients desservis sur cette route-dépôt.

2.2 | Introduction de la notion de Peak Customer

Un peak customer est défini comme le client ayant l'indice le plus élevé dans une route. Par exemple, dans le cas d'un véhicule suivant l'itinéraire (1, 2, 4, 6, 1) le peak customer sera 6. L'idée est de décomposer chaque itinéraire en deux chemins issus du dépôt et vers le peak customer. Les deux chemins sont donc disjoints à l'exception du départ (dépôt) et de l'arrivée (peak customer). Pour illustrer cette formulation nous allons nous baser sur une solution que nous avons obtenu pour l'instance statique euclidienne pour $n = 5$, de solution deux routes (1, 3, 4, 1) et (1, 2, 5, 1) avec les peak customers en gras.

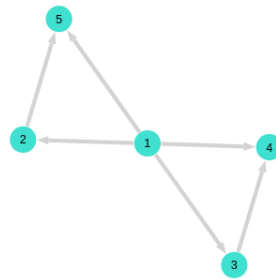


Figure 2.1: Instance $n = 5$, Euclidienne, Statique

2.3 | Formulation du problème

On écrit alors la formulation du problème, pour cela on va introduire les variables

- $p_i \in \{0, 1\}$ qui indique si le client i est un peak customer.
- $w_i \in \mathbb{N}$ qui, si $p_i = 1$, alors représente la demande totale sur la route qui dessert le client i . Elle vaut 0 sinon.
- u_i qui va représenter le plus grand indice de client visité sur le chemin dépôt-client i inclus

On a donc la formulation

$$\min \sum_{(i,j) \in A} t_{ij} x_{ij} \quad (1)$$

$$\text{s.t.} \quad \sum_{j \in V_c \cup \{1\}} x_{ji} - p_i = 1, \quad \forall i \in V_c \quad (2)$$

$$\sum_{j \in V_c} x_{ij} + p_i = 1, \quad \forall i \in V_c \quad (3)$$

$$w_i + \sum_{j \in V_c} f_{ij} = d_i + \sum_{j \in V_c} f_{ji}, \quad \forall i \in V_c \quad (4)$$

$$d_i x_{ij} \leq f_{ij} \leq (C - d_j) x_{ij}, \quad \forall (i, j) \in A \quad (5)$$

$$d_i p_i \leq t_i \leq C p_i, \quad \forall i \in V_c \quad (6)$$

$$i \leq u_i \leq i p_i + (n - 1)(1 - p_i), \quad \forall i \in V_c - \{n\} \quad (7)$$

$$u_i - u_j + (n - j - 1) x_{ij} \leq n - j - 1 \quad \forall (i, j) \in V_c - \{n\}, i \neq j \quad (8)$$

$$x_{ij} \in \{0, 1\}, f_{ij} \in \mathbb{N} \quad \forall i, j \in A_c \quad (9)$$

$$x_{1j} \in \{0, 1, 2\}, x_{j1} = 0, \quad \forall j \in V_c \quad (10)$$

$$p_i \in \{0, 1\}, t_i \in \mathbb{N}, u_i \in \mathbb{N} \quad \forall i \in V_c \quad (11)$$

Les contraintes (2)-(5) reprennent les contraintes principales de la formulation en flots et viennent assurer que l'on a bien la structure de solution décrite dans la figure 2.1, ie qu'une solution est un ensemble de routes composées par des chemins convergents vers les peak customers. On y retrouve aussi les contraintes de flots qui assurent que chaque client reçoit la quantité d_i demandée.

Les contraintes (6) et (7) viennent s'assurer que t_i et u_i soient définies comme prévu. Les contraintes (8) viennent assurer que les u_i sont croissants sur tout chemin. Si $x_{ij} = 1$ alors $u_j \geq u_i$ et sinon $u_i - u_j \leq (n - 1) - j$.

2.4 | Contraintes supplémentaires permettant de resserrer la relaxation linéaire

Les auteurs de l'article ont introduit des inégalités supplémentaires qui permettent de resserrer la relaxation linéaire

- $\sum_{i \in S_j} p_i \geq \lceil \sum_{i \in S_j} \frac{p_i}{C} \rceil, \forall j \in V_c$ Est une conséquence de notre approche pour représenter les solutions : chaque client j doit être desservi sur une route sur laquelle le peak customer est d'indice supérieur ou égal à j .
- La contrainte précédente permet également d'assurer qu'on aura au moins suffisamment de routes pour desservir l'ensemble des clients (cas $j = 2$).
- Les auteurs introduisent également ces inégalités valides:
 $u_i - u_j + (n - j - 1) x_{ij} + (n - \max(i, j) - 1) x_{ji} \leq n - j - 1, \forall i \neq j \in V_c - \{n\}$

2.5 | Application au problème robuste

Pour faciliter l'écriture nous noterons X^p l'ensemble des solutions admissibles de la formulation peak du problème. Le problème robuste s'écrit alors

$$\min_{x \in X^p} \max_{t' \in \mathcal{U}} \sum_{(i,j) \in A} t'_{ij} x_{ij} \quad (2.1)$$

$$x \in \mathcal{X} \quad (2.2)$$

Nous avons implémenté une résolution du problème par une méthode des plans coupants et par la dualisation. Il apparaît clairement que cette méthode est très intéressante du fait de la rapidité d'exécution du B and B comparé à la formulation polynomiale qui présente des symétries. Nous nous sommes cependant, rapidement rendus compte d'un problème : lorsque la solution renvoyée comportait une "one route", un véhicule ne livrant qu'un seul client j , dans notre modèle cela se traduit par deux choses : $p_j = 1$ et $x_{i,j} = 2$.

a) Problème pour les solutions présentant des "one-route"

Pour les deux approches que l'on a étudié, plans coupants (itératifs, callback) et dualisation nous avons constaté un écart entre l'optimum renvoyé et l'optimum connu (ou la meilleure solution obtenue par la formulation MTZ classique). Ces écarts apparaissent exclusivement (mais pas uniquement) lorsque la solution renvoyée présente des one-routes. Nous avons étudié en détail les différentes raisons pouvant expliquer ce résultat

- Problème de résolution du sous problème : nous avons utilisé un sac à dos fractionnaire dans la résolution par plans coupants sur la formulation classique et nous avons tenté de résoudre le problème en utilisant un solver pour le résoudre.
- Le problème apparaît plus précisément lorsque la perturbation effectuée sur un des deux arcs mais pas nécessairement l'autre. Le problème étant dans la représentation de nos solutions qui induit une contrainte plus forte lorsque l'on effectue le callback.
- Pour la dualisation, le problème est similaire mais apparaît clairement dans le cas d'une instance avec un seul client.

Nous avons alors conclu que cette formulation, qui permet d'accélérer significativement la résolution du problème (au niveau du branch and bound) en supprimant les symétries, nous donnait une borne supérieure des solutions dans le cas général.

b) Instances moyennes et grandes

Notre implémentation a révélé que la formulation Peak mise à profit pour être résolue par dualisation est particulièrement efficace et se classe parmi les trois meilleures approches que nous avons étudié pour les instances à partir de $n = 20$. Le détail est à retrouver ultérieurement dans la section résultats, et cette performance s'explique par le fait qu'on a moins de chance de retrouver des "one-outes" dans ces instances, nous évitons donc le problème mentionné précédemment.

3 | Résolution numérique

On notera RC_1 et RC_2 les coût associés respectivement aux distances $(\hat{t}_i + \hat{t}_j)_{(i,j) \in A}$ et $(\hat{t}_i \times \hat{t}_j)_{(i,j) \in A}$

3.1 | Plans coupants

Le sous-problème peut se résoudre efficacement en utilisant un algorithme de tri. En effet, résoudre le sous-problème revient à résoudre deux sous-problèmes de type sac à dos fractionnaire car aucune contrainte ne lie les variables δ^1 et δ^2 .

Pour calculer RC_1

$$\text{Pour } RC_1 = \max_{\delta^1} \sum_{(i,j) \in A} \delta_{ij}^1 (\hat{t}_i + \hat{t}_j) x_{ij}^* \quad (3.1)$$

$$\text{s.t. } \sum_{(i,j) \in A} \delta_{ij}^1 \leq T \quad (3.2)$$

$$\delta_{ij}^1 \in [0, 1], \quad \forall (i, j) \in A \quad (3.3)$$

Il suffit alors de trier les arcs sélectionnés (tel que $x_{ij}^* = 1$) par $(\hat{t}_i + \hat{t}_j)_{(i,j) \in A}$ décroissants et de sélectionner les T premiers.

On calcule RC_2 de la même manière, en comptant deux fois chaque distance perturbée (une fois pour le dernier arc si T est impair).

Nous avons implémenté cette approche de deux manières (pour les deux formulations classiques MTZ et Peak Client)

- Méthode itérative
- Branch and cut en utilisant un Lazy Callback

3.2 | Dualisation

Nous avons implémenté la dualisation de la formulation MTZ classique décrite dans la partie Modélisation et de même la dualisation de la formulation Peak Customer. Les résultats sont introduits dans la section Résultats expérimentaux.

3.3 | Métaheuristique

a) Hybrid Genetic Search

Nous avons décidé d'implémenter une méthode de recherche génétique hybride, une des métaheuristiques actuelles les plus performantes sur le CVRP statique. Ces métaheuristiques, aussi appelées algorithmes mémétiques combinent recherche génétique et recherche locale. Notre implémentation se base sur le squelette d'une implémentation réalisée par Thibault Vidal, dont le fonctionnement général est décrit dans (Vidal and al. 2012)[4] et est disponible sur Github : <https://github.com/vidalt/HGS-CVRP>

Le lecteur trouvera une description très détaillée du fonctionnement de la partie génétique de l'algorithme dans (Vidal and al. 2012)[4]. Rappelons néanmoins les grandes lignes de l'algorithme et les quelques changements apportés pour l'adapter à notre CVRP robuste.

Sélection des Parents :

Chacun des parents est sélectionné par tournoi binaire : On sélectionne au hasard deux individus de la population et on garde celui avec la meilleure fitness. La fitness dépend du coût objectif de l'individu et de considérations de diversité. Pour adapter cette sélection à notre problème robuste, il suffit de changer la contribution du coût statique par le coût robuste dans le calcul de la fitness.

Recombinaison :

Cette opération ne dépend pas du coût des parents. Nous avons donc conservé le mécanisme de recombinaison initialement présent, sans modification. HGS applique un OX crossover : l'enfant hérite d'un fragment aléatoire du premier parent puis l'on ajoute les clients non visités dans le fragment à l'aide du second parent, comme décrit dans la figure 2.1.

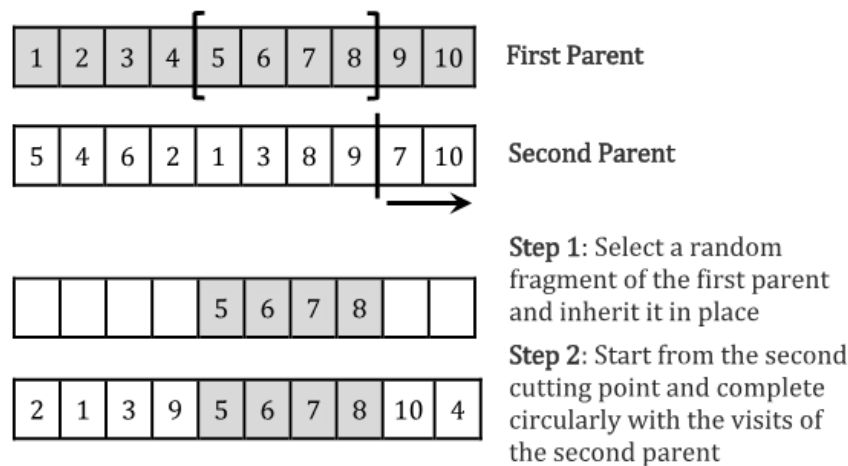


Figure 3.1: Fonctionnement OX-Crossover

Gestion de la population :

L'algorithme maintient constamment une population d'individus faisables et une population infaisable où les individus ont des tournées dépassant la capacité. Chaque sous-population est gérée de façon à contenir entre μ et $\mu + \lambda$ solutions, de telle sorte que le paramètre μ représente une taille minimale de population et le paramètre λ représente la taille d'une génération. Dès qu'une sous-population atteint $\mu + \lambda$ individus, une phase de sélection des survivants est déclenchée pour éliminer itérativement λ solutions. Ceci est réalisé en supprimant de manière itérative une solution clone (c'est-à-dire identique à une autre) si une telle solution existe, ou sinon la pire solution en termes de fitness, prenant en compte le coût robuste dans notre cas.

Recherche locale :

Après chaque croisement, les individus sont éduqués par une recherche locale. Après cette recherche locale, un individu peut toujours être infaisable. Si c'est le cas, on répare l'individu avec une probabilité de 50 %. Pour ce faire, on relance la recherche locale en augmentant fortement les facteurs de pénalités.

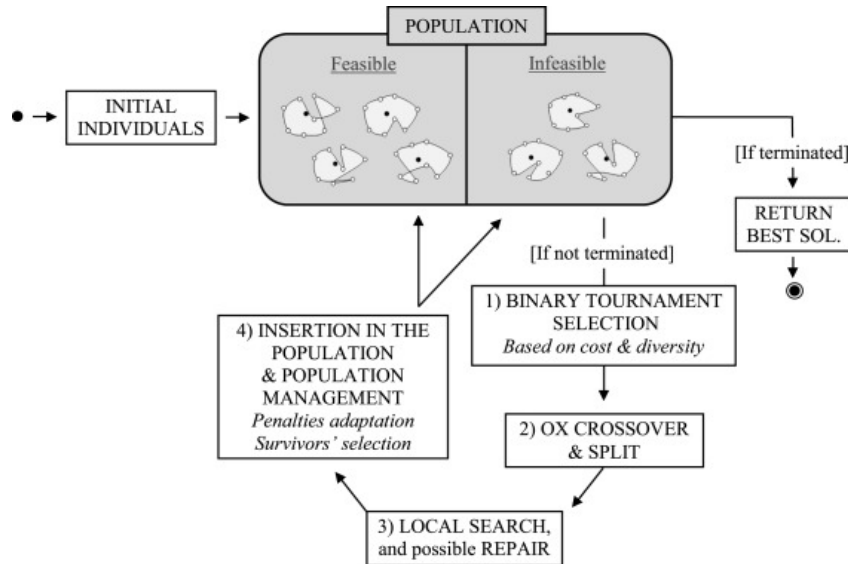


Figure 3.2: Résumé global du fonctionnement de HGS - CVRP

Détaillons maintenant notre travail d'implémentation pour la partie recherche local de HGS-CVRP.

Nous avons conservé 5 des 9 mouvements initialement présents dans l'implémentation %¹

M1 : Si U est un client, on supprime U et on l'insère après V

M2 : Si U et X sont deux clients adjacents, on les supprime et on insère (U,X) après V

M4 : Échange de U et V

M5 : Si U et X sont deux clients adjacents et V un client, on échange (U,X) et V

M7 : Si (U,X), (V,Y) sont deux paires de client adjacents d'une même route, on remplace (U,X) et (V,Y) par (U,V) et (X,Y) (2-opt intra-route)

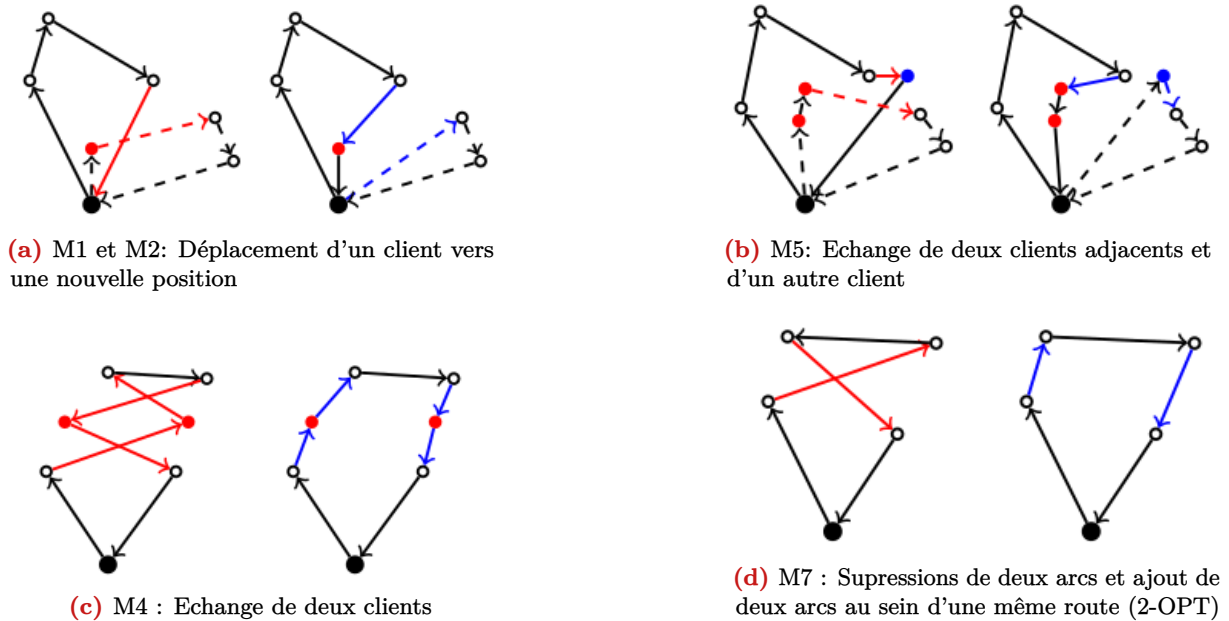


Figure 3.3: Opérateur de recherche locale (Groër 2008)[2]

¹Numération des mouvements classiques héritée de (Prins, 2004) [3]

b) Optimisation du calcul des mises à jour des coûts robustes

Pour un CVRP classique, le coût incrémental de chaque mouvement peut être calculée en $O(1)$ opérations. Dans notre cas, chaque mouvement peut également modifier le coût robuste. Même si l'on dispose d'un algorithme efficace pour calculer le coût d'une solution admissible, utiliser cet algorithme à chaque fois que l'on calcule le coût incrémental d'un mouvement rend la métaheuristique extrêmement lente sur des instances de plus de 30 clients.

Nous avons optimisé ces calculs en tirant parti du fait que chaque mouvement ne modifie qu'un nombre restreint d'arcs et donc que la majorité des arcs sélectionnés pour le coût robuste le seraient encore si le mouvement était appliqué.

Détaillons maintenant une méthode optimisée pour calculer RC_1 . L'algorithme que nous avons utilisé pour calculer RC_2 est semblable.

Considérons pour cela une liste triée par $(\hat{t}_i + \hat{t}_j)_{(i,j) \in \mathcal{A}}$ décroissante de l'ensemble de toutes les arêtes possibles, même celles non sélectionnées chez l'individu courant. Cette liste n'est triée qu'une seule fois lors de la lecture de l'instance. Notons p l'indice du dernier arc compté dans RC_1 et utilisé dans l'individu courant. On aura toujours T arcs intervenant dans le coût robuste. L'arc e_p est donc le T -ème arc avec la distance $(\hat{t}_i + \hat{t}_j)_{(i,j) \in \mathcal{A}}$ la plus élevée parmi les arcs utilisés de l'individu.

Notons D l'ensemble des arcs supprimés qui étaient comptés dans le coût robuste et N l'ensemble des arcs créés dont la position dans la liste triée est avant e_p . Remarquons alors que :

Cas 1 : $D = N = \emptyset$. Le coût robuste et la position du pointeur ne sont pas modifiés.

Cas 2 : $|D| = |N|$. On retire les coûts robustes des arcs de D et on ajoute ceux de N . Le pointeur p est potentiellement décalé vers la gauche si e_p est supprimé.

Cas 3 : $|D| > |N|$. On ajoute forcément tous les coûts robustes des arcs de N et on supprime tous les coûts robustes des arcs de D . Il faut rajouter les coûts robustes de $|D| - |N|$ arcs qui étaient auparavant après e_p . Le pointeur p est donc décaler vers la droite.

Cas 4 : $|D| < |N|$. On supprime les coûts robustes des arcs de D et on rajoute les coûts des $|N| - |D|$ arcs avec les coûts les plus élevées. Le pointeur p est décalé vers la gauche.

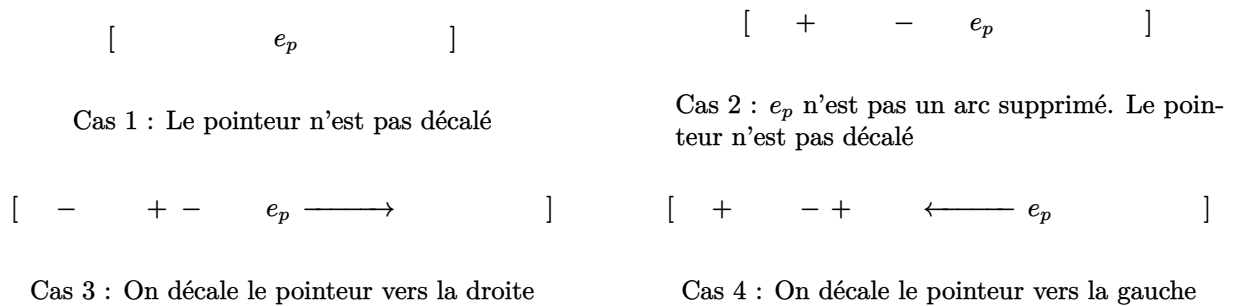


Figure 3.4: Visualisation des règles de mise à jour du pointeur p . Les arcs $+$ et $-$ représentent respectivement les arcs de N et D

Pour le calcul du coût robuste 2, si T est pair, T^2 l'est aussi. On utilise l'algorithme précédant en ajoutant et retirant 2 fois les coûts robustes des arcs que l'on ajoute ou supprime. Si T est impair, T^2 l'est aussi. On adapte l'algorithme en prenant garde au fait que le coût robuste du dernier arc sélectionné ne contribuera qu'une seule fois au coût robuste.

Cette méthode permet d'éviter de recalculer le coût robuste en parcourant et triant à chaque fois les listes des arcs sélectionnés. Cette version nous a permis de réaliser des milliers d'itérations en temps raisonnable (<5 minutes) pour les instances de plus de 30 clients.

Instances	Temps de calcul moyen pour 100 itérations (s)		
	HGS-VRP (Statique)	Version Optimisée	Version Naïve
n_50-euclidean_true	<0.5	3.5	12
n_80-euclidean_true	<0.5	3	70
n_100-euclidean_true	<1	4	> 200

Table 3.1: Comparaison des temps de calcul moyen pour différentes instances.

3.4 | Génération de colonnes

Décrivons un schéma de génération de colonnes appliquée à la dualisation de notre problème robuste.

Si on note R l'ensemble des routes, le problème admet la reformulation non compacte suivante :

$$(RRMP) \quad \min \sum_{r \in R} c_r y_r + \lambda_1 T + \lambda_2 T^2 + \sum_{(i,j) \in A} \mu_{ij}^1 + 2 \sum_{(i,j) \in A} \mu_{ij}^2 \quad (3.4)$$

s.t

$$\sum_{r \in R: i \in r} y_r = 1, \quad \forall i \in V \setminus \{0\}, \quad (3.5)$$

$$\mu_{ij}^1 + \lambda_1 \geq (\hat{t}_i + \hat{t}_j) \sum_{r \in R: (i,j) \in r} y_r \quad \forall (i,j) \in A \quad (3.6)$$

$$\mu_{ij}^2 + \lambda_2 \geq \hat{t}_i \hat{t}_j \sum_{r \in R: (i,j) \in r} y_r \quad \forall (i,j) \in A \quad (3.7)$$

$$y_r \in \{0, 1\} \quad \forall r \in R \quad (3.8)$$

$$\mu_{ij}^1, \mu_{ij}^2 \geq 0 \quad \forall (i,j) \in A \quad (3.9)$$

$$\lambda_1, \lambda_2 \geq 0 \quad (3.10)$$

$$(3.11)$$

Le dual de la relaxation de ce problème est donnée par :

$$(RRMP \text{ Dual}) \quad \max \sum_{i \in V \setminus \{0\}} \pi_i \quad (3.12)$$

s.t

$$\sum_{i \in r} \pi_i - \sum_{(i,j) \in r} z_{i,j}^1 (\hat{t}_i + \hat{t}_j) - \sum_{(i,j) \in r} z_{i,j}^2 (\hat{t}_i \hat{t}_j) \leq c_r \quad \forall r \in R, \quad (3.13)$$

$$\sum_{(i,j) \in A} z_{ij}^1 \leq T \quad (3.14)$$

$$\sum_{(i,j) \in A} z_{ij}^2 \leq T^2 \quad (3.15)$$

$$z_{ij}^1 \leq 1 \quad \forall (i,j) \in A \quad (3.16)$$

$$z_{ij}^2 \leq 2 \quad \forall (i,j) \in A \quad (3.17)$$

$$z_{ij}^1, z_{ij}^2 \geq 0 \quad \forall (i,j) \in A \quad (3.18)$$

$$(3.19)$$

Pour π^* , z^1 et z^2 solution optimale duale, le problème de pricing admet donc la formulation suivante :

$$(PCTSP) \quad \min \sum_{(i,j) \in E} (c_{ij} + z_{i,j}^*{}^1(\hat{t}_i + \hat{t}_j) + z_{i,j}^*{}^2 \hat{t}_i \hat{t}_j) x_{ij} - \sum_{i \in V'} \pi_i^* z_i \quad (3.20)$$

s.t

$$z_0 = 1 \quad (3.21)$$

$$\sum_{(i,j) \in E} x_{ij} = 2 z_i \quad \forall i \in V \quad (3.22)$$

$$\sum_{i \in V} d_i z_i \leq C \quad (3.23)$$

$$\sum_{(i,j) \in S} x_{ij} \leq \sum_{i \in S} z_i - 1 \quad \forall S \subseteq V', |S| \geq 2, \quad (3.24)$$

$$x_{ij}, z_k \in \{0, 1\}, \quad (i, j) \in E, k \in V. \quad (3.25)$$

On parle dans la littérature de price-collecting TSP (ou de plus court chemin sous contrainte de manière équivalente). Il s'agit d'un TSP modifié où il n'est pas obligatoire de passer par tous les noeuds. Nous avons choisi de résoudre les sous-problèmes par branch and cut i.e avec élimination des sous-tours par callback.

Après avoir obtenue une solution fractionnaire de la relaxation de RRMP, on construit une solution valide en résolvant RRMP sur l'ensemble de routes obtenu durant la génération de colonnes. La méthode n'est donc pas exacte.

Ci-dessous, le fonctionnement résumé de la méthode :

Algorithm 1 Génération de colonnes pour Robust CVRP

Initialisation:

Générer un ensemble initial de routes à l'aide d'une heuristique Ω : LKH avec 2-opt et 3-opt dans notre cas

Répéter

Résoudre le problème maître restreint relaché (Robust RMP) $y_r \in [0, 1]$

Récupérer les valeurs duales π^* , z^1 et z^2 des contraintes du problème maître.

Sous-problème:

Résoudre un Price Collecting TSP

avec coûts modifiés $\tilde{c}_{ij} = c_{ij} + z_{i,j}^*{}^1(\hat{t}_i + \hat{t}_j) + z_{i,j}^*{}^2 \hat{t}_i \hat{t}_j - \pi_i$ sur les arcs.

si une route de coût réduit négatif est trouvée **alors**

Ajouter cette route à Ω

sinon

Break (optimum atteint)

fin si

Jusqu'à convergence ou limite de temps atteinte

Post-traitement Heuristique:

Résoudre le problème maître avec $y_r \in \{0, 1\}$ pour les routes de Ω

4 | Résultats expérimentaux

- Nous avons codé la métaheuristique en C++.
- Toutes les autres méthodes ont été codé en Julia. Nous avons utilisé JuMP et le solveur Gurobi.
- L'ensemble des tests ont été réalisé avec une CPU Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz.

4.1 | Petites Instances ($n \leq 30$)

Les méthodes exactes nous ont permis de connaître les valeurs optimales pour $n = 6$ à 20 pour les instances euclidiennes et de $n = 6$ à 30 pour les instances non euclidiennes. La métaheuristique trouve l'ensemble des solutions optimales pour ces instances.

Comparons donc le temps d'exécution des méthodes exactes et de Robust-HGS pour trouver l'optimale, avec un temps d'exécution maximum de 3 minutes :

... : Instance non résolue en moins de 3 minutes.

Instance	MTZ-CP-iteratif	MTZ-B&C	Dualisation	Métaheuristique	Opt	PR
n_6-euclidean_true	0.65	0.05	0.03	0.06	3087	2.15%
n_7-euclidean_true	0.45	0.06	0.09	0.11	3132	5.00%
n_8-euclidean_true	0.14	0.05	0.02	0.15	3776	5.65%
n_9-euclidean_true	3.71	0.09	0.15	0.09	4181	18.4%
n_10-euclidean_true	2.06	0.11	0.24	0.24	5423	12.1%
n_11-euclidean_true	72.4	0.74	0.40	0.56	7142	14.8%
n_12-euclidean_true	...	2.69	0.33	0.30	6364	11.8%
n_13-euclidean_true	...	1.89	0.42	0.34	5971	19.7%
n_14-euclidean_true	...	...	38.37	0.45	7669	16.8%
n_15-euclidean_true	...	6.21	7.76	0.32	6140	4.85%
n_16-euclidean_true	...	27.85	4.84	0.59	9144	8.22%
n_17-euclidean_true	...	85.56	42.26	1.39	6698	6.72%
n_18-euclidean_true	...	...	55.79	0.91	10154	20.6%
n_19-euclidean_true	84	9.77	11.35	0.98	6987	4.0%
n_20-euclidean_true	...	...	55.29	0.94	9098	12.6%

Table 4.1: Comparaison temps d'exécution(s) pour trouver la solution optimale - instance euclidiennes

Instance	MTZ-CP-iteratif	MTZ-B&C	Dualisation	Métaheuristique	Opt	PR
n_6-euclidean_false	0.57	0.02	0.0061	0.07	3307	22.7 %
n_7-euclidean_false	0.01	0.01	0.0070	0.05	2066	10.1%
n_8-euclidean_false	0.05	0.06	0.0170	0.11	3270	28.5%
n_9-euclidean_false	0.05	0.07	0.023	0.12	3113	13.3%
n_10-euclidean_false	0.60	0.10	0.17	0.21	2684	18.4%
n_11-euclidean_false	0.79	0.42	0.073	0.21	3899	25.4%
n_12-euclidean_false	0.70	0.22	0.11	0.24	3389	13.0%
n_13-euclidean_false	0.27	0.16	0.089	0.28	2682	17.3%
n_14-euclidean_false	23.5	2.15	0.29	0.52	5090	26.9%
n_15-euclidean_false	0.15	0.08	0.037	0.35	2126	12.8%
n_16-euclidean_false	...	...	0.96	0.50	849	57.2%
n_17-euclidean_false	...	90	0.16	0.33	619	66.5%
n_18-euclidean_false	...	...	—	4.20	1300	68.3%
n_19-euclidean_false	...	...	6.06	0.68	1182	76.5%
n_20-euclidean_false	...	...	1.95	1.55	668	69.9%
n_25-euclidean_false	...	...	...	...	1993	79.2%
n_30-euclidean_false	...	...	1.95	1.55	1807	83.3%

Table 4.2: Comparaison des temps d'exécution (s) pour trouver la solution optimale - instances non euclidiennes

La comparaison est plus aisée sur les diagrammes de performances suivants :

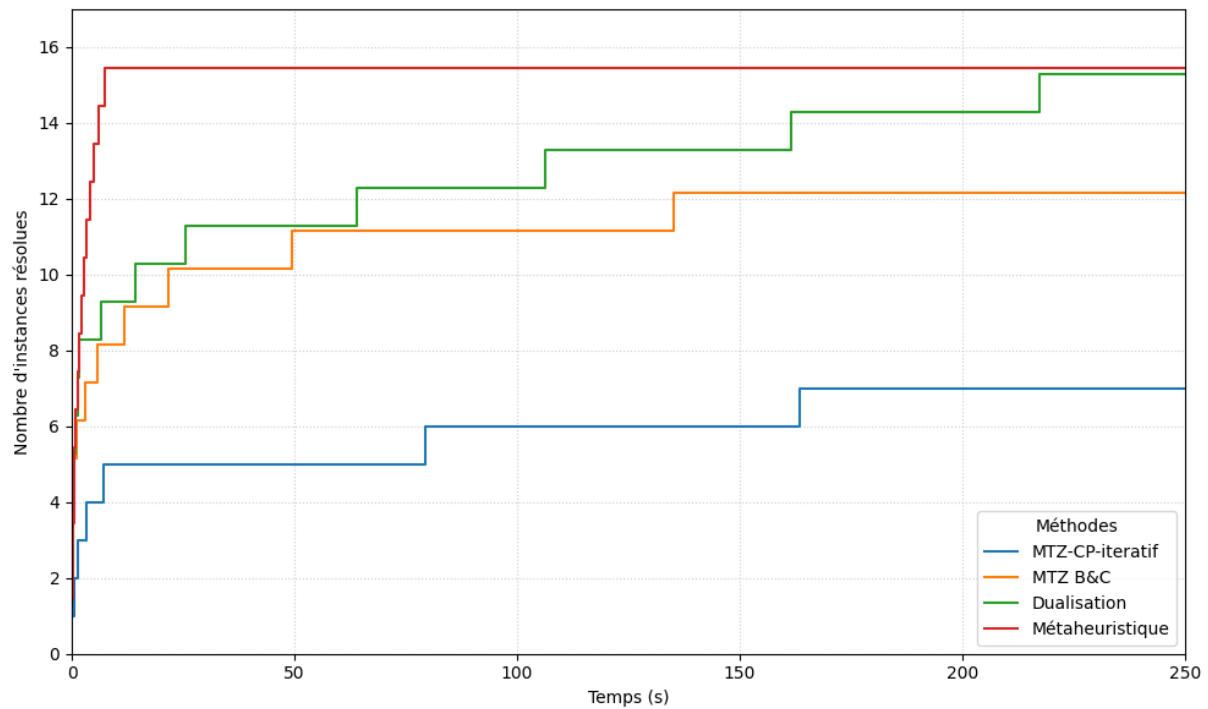


Figure 4.1: Performances - Instance euclidiennes $n = 6$ à 20

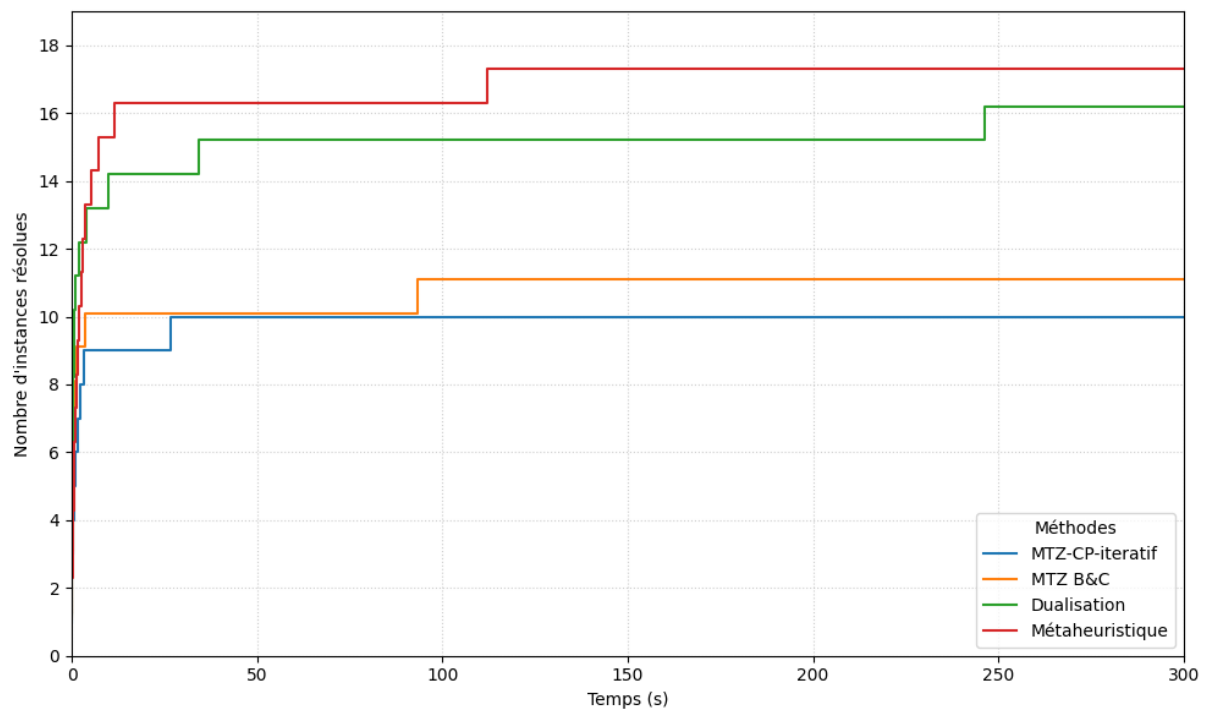


Figure 4.2: Performances - Instance non euclidiennes $n = 6$ à 30

Remarquons que :

- La métaheuristique est la méthode la plus efficace sur ces instances.
- Le Branch & Cut est plus efficace que les plans coupants itératifs.
- La dualisation MTZ fonctionne mieux sur les instances non euclidiennes.

4.2 | Grandes instances

Ci-dessous sont comparées les performances des trois meilleures méthodes que nous avons testé :

* : Instance avec optimalité prouvée par une des méthodes exactes. La colonne Gap correspond au gap par rapport à la meilleure solution connue.

Instance	Peak Dual		MTZ Dual		Robust HGS	
	Cost	Gap (%)	Cost	Gap (%)	Cost	Gap (%)
n_25-euclidean_true	9375	0.00	9375	0.00	9375	0.00
n_30-euclidean_true	14727	0.91	14836	1.66	14594	0.00
n_35-euclidean_true	14543	0.01	14571	0.20	14542	0.00
n_40-euclidean_true	10232	0.00	10830	5.83	10232	0.00
n_45-euclidean_true	14890	1.69	15058	2.83	14643	0.00
n_50-euclidean_true	9718	0.35	9822	1.43	9684	0.00
n_55-euclidean_true	12400	8.42	11958	4.57	11436	0.00
n_60-euclidean_true	15641	0.81	15893	2.43	15516	0.00
n_65-euclidean_true	17894	1.07	18288	3.29	17705	0.00
n_70-euclidean_true	12607	9.89	12346	7.61	11474	0.00
n_75-euclidean_true	15555	5.67	15888	7.95	14718	0.00
n_80-euclidean_true	16018	5.66	16599	9.48	15160	0.00
n_85-euclidean_true	14074	7.48	15707	19.86	13097	0.00
n_90-euclidean_true	36020	3.78	36391	4.86	34705	0.00
n_95-euclidean_true	28921	3.62	28813	3.24	27910	0.00
n_100-euclidean_true	11375	2.95	11829	7.07	11049	0.00

Table 4.3: Comparaison de la qualité des solutions et gap à Tmax = 5 minutes

- Avec un gap moyen de 3.27% (contre 5.14 % pour MTZ Dual) , la méthode Peak Dual est meilleure pour la majorité des instances euclidiennes.
- La métaheuristique obtient la meilleure solution pour toute les instances euclidiennes.

La méthode Peak Dual n'est pas incluse car elle ne fonctionne que sur les instances symétriques.

Instance	MTZ Dual		Robust HGS	
	Cost	Gap	Cost	Gap
n_25-euclidean_false	1993*	0.00%	1993	0.00%
n_30-euclidean_false	1807*	0.00%	1807	0.00%
n_35-euclidean_false	1783*	0.00%	1783	0.00%
n_40-euclidean_false	2855	0.00%	2859	0.14%
n_45-euclidean_false	1280	0.00%	1280	0.00%
n_50-euclidean_false	884	0.23%	882	0.00%
n_55-euclidean_false	2177*	0.00%	2276	4.55%
n_60-euclidean_false	1821	3.06%	1767	0.00%
n_65-euclidean_false	2042	0.00%	2100	2.84%
n_70-euclidean_false	2668	1.72%	2623	0.00%
n_75-euclidean_false	2294	0.00%	2451	6.84%
n_80-euclidean_false	657	0.00%	766	16.59%
n_85-euclidean_false	2222	0.00%	2246	1.08%
n_90-euclidean_false	2968	0.24%	2961	0.00%
n_95-euclidean_false	3228	0.00%	3284	1.73%
n_100-euclidean_false	2953	0.00%	3055	3.45%

Table 4.4: Comparaison de la qualité des solutions et gap à Tmax = 5 minutes

- La métaheuristique n'est pas tout le temps meilleure que la dualisation pour les instances non euclidiennes. Nous pensons que cela est du au fait que nous n'avons pas inclus les 2-opt inter routes,

qui introduisent la modification du sens de parcours de portion de routes. Ces mouvements étaient initialement présents dans HGS-CVRP mais nous n'avons pas eu le temps des les adapter. Une piste d'amélioration serait donc d'ajouter ces mouvements. Cela s'explique également par le fait que la méthode dual fonctionne mieux sur les instances non euclidiennes.

4.3 | Retour sur la méthode de génération de colonnes

Voici les résultats obtenus sur les instances euclidiennes de $n = 6$ à 30 pour la génération de colonne :

Instance	Time(s)	Num Itérations	Cost	MSC	Gap
n_6-euclidean_true	0.21	2	3111	3087*	0.78%
n_7-euclidean_true	0.33	4	3142	3132*	0.32%
n_8-euclidean_true	0.44	1	3776	3776*	0.00%
n_9-euclidean_true	0.50	2	4181	4181*	0.00%
n_10-euclidean_true	0.57	4	5423	5423*	0.00%
n_11-euclidean_true	0.60	2	7142	7142*	0.00%
n_12-euclidean_true	0.70	7	6406	6364*	0.66%
n_13-euclidean_true	1.19	15	5971	5971*	0.00%
n_14-euclidean_true	0.91	10	7669	7669*	0.00%
n_15-euclidean_true	0.84	9	6158	6140*	0.29%
n_16-euclidean_true	0.63	1	9144	9144*	0.00%
n_17-euclidean_true	2.9	25	6836	6698*	2.06%
n_18-euclidean_true	2.9	25	10179	10154*	0.25%
n_19-euclidean_true	2.25	24	7188	6987*	2.88%
n_20-euclidean_true	3.0	20	9419	9098*	3.53%
n_25-euclidean_true	7.48	38	9626	9375	2.68%
n_30-euclidean_true	12.0	36	14883	14594	1.98%

Table 4.5: Résultats - Génération de colonnes

Sous sa forme actuelle, la méthode n'est pas tractable sur de plus grandes instances. Le PCTSP est un problème NP-difficile et la résolution par Branch & Cut devient lente au dela de 40 noeuds.

Pistes d'amélioration :

- La méthode converge en moins de 3 secondes sur les petites instances ($n \leq 20$). On aurait donc pu rendre la méthode exacte en implémentant un Branch & Price.
- Il existe des algorithmes pseudo-polynomiaux exactes (programmation dynamique) et des heuristiques efficaces pour résoudre le problème de pricing. Ces méthodes permettent également de renvoyer plusieurs routes à ajouter, accélérant la convergence. On peut en trouver des implémentations optimisées en C sur <https://github.com/torressa/cspy>. C'est notamment ce qui est utilisé dans la librairie python `vrpy`.

References

- [1] S.L. Gadegaard and J. Lysgaard. “A symmetry-free polynomial formulation of the capacitated vehicle routing problem”. In: *Discrete Applied Mathematics* 296 (2021). 16th Cologne–Twente Workshop on Graphs and Combinatorial Optimization (CTW 2018), pp. 179–192. ISSN: 0166-218X. DOI: <https://doi.org/10.1016/j.dam.2020.02.012>. URL: <https://www.sciencedirect.com/science/article/pii/S0166218X20300779>.
- [2] Chris Groër, Bruce Golden, and Edward Wasil. “A library of local search heuristics for the vehicle routing problem”. In: *Mathematical Programming Computation* 2 (June 2010), pp. 79–101. DOI: [10.1007/s12532-010-0013-5](https://doi.org/10.1007/s12532-010-0013-5).
- [3] Christian Prins. “Prins, C.: A simple and effective evolutionary algorithm for the vehicle routing problem. *Computer & Operations Research* 31(12), 1985–2002”. In: *Computers & Operations Research* 31 (Oct. 2004), pp. 1985–2002. DOI: [10.1016/S0305-0548\(03\)00158-8](https://doi.org/10.1016/S0305-0548(03)00158-8).
- [4] Thibaut Vidal et al. “A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems”. In: *Operations Research* 60 (June 2012), pp. 611–624. DOI: [10.1287/opre.1120.1048](https://doi.org/10.1287/opre.1120.1048).